

## **Lekcja - przełożony sprawdzian LEKCJA jest zrobiona na 26 marca**

### **Temat: is oraz ==, funkcja rekurencyjna. Typy mutowalne i niemutowalne, inkrementacja, dekrementacja. Try-except-else-finally, raise i custom exceptions**

Operator **==** porównuje wartość. Inaczej mówiąc **sprawdza, czy wartości dwóch obiektów są takie same. Nie bierze pod uwagę, czy obiekty są przechowywane w tym samym miejscu w pamięci – liczy się tylko zawartość.** Jest to porównanie semantyczne, oparte na metodzie `__eq__` obiektu.

Przykłady

```
a = 5
b = 5
print(a == b)
```

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)
```

Operator **is** porównanie obiektu w pamięci. Inaczej mówiąc **sprawdza, czy dwie zmienne wskazują na ten sam obiekt w pamięci.**

Przykłady

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a is b)
```

```
a = [1, 2, 3]
b = a
print(a is b)
```

Python ma mechanizm zwany **internowaniem małych liczb całkowitych** (integer caching).

- Dla liczb z zakresu **-5 do 256** Python **przechowuje jedną wspólną instancję**
- Czyli np. liczba 10 istnieje tylko raz w pamięci

```
a = 10
b = int("10")
print(a is b) # True
```

```
c = 256 # -5 do 256
d = int("256")
print(c is d) # True
```

```
e = 1000
f = int("1000")
print(e is f) # False
```

### Przykład – list

```
a = [1, 2, 3]
b = [1, 2, 3]
```

```
print(a == b)
print(a is b)
```

### Przykład – dict

```
a = {"name": "Jan"}
b = {"name": "Jan"}
```

```
print(a == b)
print(a is b)
```

### Przykład – set

```
a = {1, 2, 3}
b = {1, 2, 3}
```

```
print(a == b)
print(a is b)
```

### Przykład – is – None

```
x = None
```

```
if x is None:
    print("brak wartości")
```

**Rekurencja** to technika programowania, w której **funkcja wywołuje samą siebie, aby rozwiązać problem. Zamiast używać pętli** (jak for czy while), **funkcja dzieli problem na mniejsze podproblemy**, aż dojdzie do prostego przypadku, który można rozwiązać bezpośrednio. Kluczowe elementy funkcji rekurencyjnej:

- **Przypadek bazowy (base case):** Warunek, który kończy rekurencję. Bez niego funkcja będzie się wywoływać w nieskończoność, co spowoduje błąd (RecursionError w Pythonie, bo przekroczony zostanie limit rekurencji, domyślnie ok. 1000 wywołań).
- **Przypadek rekurencyjny (recursive case):** Część, w której funkcja wywołuje siebie z mniejszymi argumentami, zbliżając się do przypadku bazowego.

Rekurencja jest przydatna w problemach, które mają strukturę drzewiastą lub dzielą się na podproblemy, np. obliczanie silni, przeszukiwanie drzew, sortowanie (jak quicksort) czy generowanie fraktali.

### Przykład: Obliczanie silni

```
def silnia(n):
    if n == 0 or n == 1: # Przypadek bazowy: 0! = 1, 1! = 1
        return 1
    else: # Przypadek rekurencyjny
        return n * silnia(n - 1)
```

Jak to działa?

- Dla `silnia(5)`: Wywołuje  $5 * \text{silnia}(4)$
- `silnia(4)`: Wywołuje  $4 * \text{silnia}(3)$
- `silnia(3)`: Wywołuje  $3 * \text{silnia}(2)$
- `silnia(2)`: Wywołuje  $2 * \text{silnia}(1)$
- `silnia(1)`: Zwraca **1** (przypadek bazowy)
- Teraz "wspinamy się" z powrotem:  $2 * 1 = 2$ ,  $3 * 2 = 6$ ,  $4 * 6 = 24$ ,  $5 * 24 = 120$ .

## Typy mutowalne i niemutowalne

W Pythonie typy danych dzielą się na **mutowalne (mutable)** i **niemutowalne (immutable)** w zależności od tego, czy można je modyfikować po utworzeniu. To kluczowa koncepcja, bo wpływa na to, jak działają przypisania, funkcje i struktury danych (np. listy jako klucze w słownikach nie działają, bo są mutowalne).

- **Niemutowalne (immutable):** Po stworzeniu obiektu **nie można zmienić jego wartości. Zamiast modyfikować, Python tworzy nowy obiekt.** To bezpieczniejsze w kontekstach wielowątkowych i jako klucze w słownikach (muszą być hashable).
- **Mutowalne (mutable):** Można zmieniać zawartość obiektu **bezpośrednio, bez tworzenia nowego.** To przydatne dla dynamicznych struktur, ale może prowadzić do nieoczekiwanych efektów (np. modyfikacja listy w funkcji wpływa na oryginalną listę).

Użyję tabeli do porównania powszechnych typów:

| Typ danych                               | Mutowalny? | Przykłady użycia                                             | Uwagi                                                 |
|------------------------------------------|------------|--------------------------------------------------------------|-------------------------------------------------------|
| <b>int</b> (liczba całkowita)            | Nie        | <code>x = 5; x = 6</code> (tworzy nowy obiekt)               | Zmiana wartości to nowe przypisanie.                  |
| <b>float</b> (liczba zmiennoprzecinkowa) | Nie        | <code>y = 3.14</code>                                        | Jak int, niezmienny.                                  |
| <b>str</b> (ciąg znaków)                 | Nie        | <code>s = "hello"; s.upper()</code> (zwraca nowy string)     | Metody jak <code>replace()</code> tworzą nowy obiekt. |
| <b>tuple</b> (krotka)                    | Nie        | <code>t = (1, 2, 3)</code>                                   | Nie można dodać/usunąć elementów.                     |
| <b>frozenset</b> (zamrożony zbiór)       | Nie        | <code>fs = frozenset([1, 2])</code>                          | Jak set, ale niezmienny.                              |
| <b>list</b> (lista)                      | Tak        | <code>l = [1, 2]; l.append(3)</code> (modyfikuje oryginalną) | Można dodawać, usuwać, sortować.                      |

|                                   |     |                                       |                                     |
|-----------------------------------|-----|---------------------------------------|-------------------------------------|
| <b>dict</b> (słownik)             | Tak | d = {'a': 1}; d['b'] = 2              | Dodawanie/usuwanie kluczy/wartości. |
| <b>set</b> (zbiór)                | Tak | s = {1, 2}; s.add(3)                  | Dynamiczne dodawanie/usuwanie.      |
| <b>bytearray</b> (tablica bajtów) | Tak | ba = bytearray(b'abc');<br>ba[0] = 65 | Modyfikowalna wersja bytes.         |

## Różnice w praktyce

- **Przypisanie i modyfikacja:** Dla typów niemutowalnych, jak string, s = "hello"; s = s + " world" tworzy nowy string. Dla list: l = [1]; l += [2] modyfikuje istniejącą listę.
- **Funkcje i argumenty:** Jeśli przekażesz mutowalny obiekt do funkcji, zmiany w funkcji wpłyną na oryginalny obiekt (przekazywanie przez referencję). Dla niemutowalnych – nie.
- **Hashowalność:** Tylko niemutowalne typy mogą być kluczami w słownikach lub elementami w setach, bo ich hash nie zmienia się.

## Przykład kodu

```
# Niemutowalny: string
s = "hello"
s2 = s # s2 wskazuje na ten sam obiekt
s = s.upper() # Tworzy nowy obiekt, s2 zostaje "hello"
print(s) # "HELLO"
print(s2) # "hello"
```

```
# Mutowalny: lista
l = [1, 2]
l2 = l # l2 wskazuje na tę samą listę
l.append(3) # Modyfikuje oryginalną listę
print(l) # [1, 2, 3]
print(l2) # [1, 2, 3] – też zmienione!
```

## Python nie ma ++ i --

Python jest zaprojektowany, aby być prostym i czytelnym (zgodnie z "Zen of Python" – "Readability counts"). Operatory ++ i -- są skrótami, które mogą być mylące, bo działają różnie w zależności od kontekstu (np. pre-inkrementacja ++x vs. post-inkrementacja x++). **W Pythonie preferuje się jawne operacje, jak x = x + 1 lub x += 1, co jest łatwiejsze do zrozumienia na pierwszy rzut oka.**

```
x = 5
x++ # Błąd: SyntaxError: invalid syntax
```

## W JavaScript np.:

**Pre-inkrementacja/dekrementacja:** Najpierw modyfikuje zmienną, potem zwraca nową wartość.

- ++x: Zwiększ x o 1, zwróć nowe x.
- --x: Zmniejsz x o 1, zwróć nowe x.

**Post-inkrementacja/dekrementacja:** Najpierw zwraca bieżącą wartość, potem modyfikuje zmienną.

- x++: Zwróć bieżące x, potem zwiększ o 1.

- `x--`: Zwróć bieżące `x`, potem zmniejsz o 1.

Python **nie ma wbudowanych operatorów `++` ani `--`**. Zamiast tego używa się operatorów przypisania złożonego: `+=` dla inkrementacji i `-=` dla dekrementacji. To proste i czytelne, ale wymaga jawnego zapisu. Te operatory działają na zmiennych liczbowych (`int`, `float`), ale też na niektórych kolekcjach (np. listy z `+=` do dodawania elementów).

## Podstawowe przykłady

*# Inkrementacja*

```
x = 5
x += 1  # To samo co x = x + 1
print(x)  # Wyjście: 6
```

*# Dekrementacja*

```
y = 10
y -= 2  # To samo co y = y - 2
print(y)  # Wyjście: 8
```

*# Można używać z innymi wartościami*

```
z = 3.5
z += 0.5  # Inkrementacja o 0.5
print(z)  # Wyjście: 4.0
```

*# Dla stringów lub list (rozszerzone użycie +=)*

```
s = "Witaj"
s += " świecie!"  # Konkatenacja
print(s)  # Wyjście: Witaj świecie!
```

```
lista = [1, 2]
lista += [3]  # Dodawanie elementu
print(lista)  # Wyjście: [1, 2, 3]
```

Inkrementacja i dekrementacja często pojawiają się w pętlach, aby kontrolować liczniki lub iterować wstecz. Python preferuje pętle **for** z **range()**, które automatycznie obsługują inkrementację, ale w **while** musisz to robić ręcznie.

### 1. Pętla for z inkrementacją (używając range):

- a. `range(start, stop, step)` automatycznie inkrementuje (`step=1` domyślnie) lub dekrementuje (`step=-1`).

*# Inkrementacja w pętli for (od 0 do 4)*

```
for i in range(5):  # i zaczyna od 0, inkrementuje o 1 co iterację
    print(i)  # Wyjście: 0 1 2 3 4
```

*# Dekrementacja w pętli for (od 5 do 1)*

```
for j in range(5, 0, -1):  # Start=5, stop=0 (nie inkluzyny), step=-1
    print(j)  # Wyjście: 5 4 3 2 1
```

*# Ręczna inkrementacja w pętli for (rzadziej używana, ale możliwe)*

```
licznik = 0
```

```
for _ in range(3): # Pętla 3 razy
    licznik += 2 # Inkrementacja o 2
    print(licznik) # Wyjście: 2 4 6
```

### Pętla while z inkrementacją/dekrementacją:

- Tutaj operatory += i -= są kluczowe, bo kontrolujesz warunek ręcznie.

```
# Inkrementacja w while (licznik rosnący)
licznik = 0
while licznik < 5:
    print(licznik) # Wyjście: 0 1 2 3 4
    licznik += 1 # Inkrementacja
```

```
# Dekrementacja w while (licznik malejący)
licznik = 10
while licznik > 0:
    print(licznik) # Wyjście: 10 9 8 ... 1
    licznik -= 1 # Dekrementacja
```

```
# Przykład z warunkiem i inkrementacją o zmienną wartość
suma = 0
i = 1
while suma < 20:
    suma += i # Inkrementacja o i (1, potem 2, itd.)
    i += 1 # Inkrementacja i
    print(suma) # Wyjście: 1 3 6 10 15 21 (zatrzyma się przed 21, bo warunek)
```

## Wyjątki w Pythonie: Try-except-else-finally, raise i custom exceptions

Wyjątki to mechanizm, który pozwala programowi radzić sobie z błędami w czasie wykonania (runtime errors), np. dzielenie przez zero, brak pliku czy nieoczekiwane dane. Zamiast crashować program, możesz "złapać" błąd i obsłużyć go elegancko. To kluczowe dla stabilnego kodu. Python ma wbudowane wyjątki (np. `ZeroDivisionError`, `FileNotFoundError`), ale możesz też tworzyć własne. Podstawowa struktura to blok try-except, z opcjonalnymi else i finally.

### Podstawowa struktura: try-except

- try:** W tym bloku umieszczasz kod, który może spowodować błąd.
- except:** Łapie wyjątek i obsługuje go. Możesz sprecyzować typ wyjątku (np. `except ValueError:`) lub złapać wszystkie (`except:` – ale to niezalecane, bo maskuje błędy).

### Przykład

```
try:
    liczba = int(input("Podaj liczbę: ")) # Może rzucić ValueError, jeśli nie liczba
    wynik = 10 / liczba # Może rzucić ZeroDivisionError
```

```

    print(wynik)
except ValueError:
    print("To nie jest liczba!")
except ZeroDivisionError:
    print("Nie dziel przez zero!")

```

### Dodatkowe bloki: else i finally

- **else:** Wykonuje się tylko, jeśli w try NIE wystąpił żaden wyjątek. Przydatne do kodu, który ma działać po sukcesie.
- **finally:** Zawsze się wykonuje, niezależnie od tego, czy był wyjątek czy nie. Idealne do czyszczenia zasobów (np. zamykanie plików).

```

try:
    a = int(input("Podaj pierwszą liczbę: ")) # Może rzucić ValueError
    b = int(input("Podaj drugą liczbę: ")) # Może rzucić ValueError
    wynik = a / b # Może rzucić ZeroDivisionError
except ValueError:
    print("Jedna z wartości nie jest liczbą całkowitą!")
except ZeroDivisionError:
    print("Nie można dzielić przez zero!")
except Exception as e:
    print(f"Nieoczekiwany błąd: {e}")
else:
    print(f"Wynik dzielenia: {wynik}")
    print("Obliczenia zakończone sukcesem.")
finally:
    print("Program zakończył przetwarzanie wejścia – zawsze to się wyświetli.")

```

**Raise: Rzucanie wyjątków** raise pozwala ręcznie "rzucić" wyjątek, np. gdy chcesz przerwać wykonanie przy niepoprawnych danych. Możesz raise'ować wbudowany wyjątek lub własny.

Przykład:

```

def sprawdz_wiek(wiek):
    if wiek < 18:
        raise ValueError("Jesteś za młody!") # Rzuca wyjątek z komunikatem
    return "OK"

try:
    sprawdz_wiek(15)
except ValueError as e:
    print(e) # Wyjście: Jesteś za młody!

```

**Custom exceptions: Własne wyjątki** Możesz tworzyć własne klasy wyjątków, dziedzicząc po Exception (lub podklasach jak ValueError). To przydatne w dużych projektach, by mieć specyficzne błędy. Przykład tworzenia i użycia:

```

class MojBład(Exception): # Dziedziczy po Exception
    def __init__(self, wiadomosc="To mój custom błąd!"):
        self.wiadomosc = wiadomosc
        super().__init__(self.wiadomosc)

def funkcja():
    raise MojBład("Coś poszło nie tak.")

try:
    funkcja()
except MojBład as e:
    print(e) # Wyjście: To mój custom błąd!

```

## Lekcja10

### Temat: Metody dla list (list), zbiorów (set) i słowników (dict)

Lista to **uporządkowana kolekcja elementów**, która **pozwala na duplikaty** i ma **indeksy**.

```
numbers = [1, 2, 3]
```

#### **append(x)**

Dodaje element **na koniec listy**.

```
numbers.append(4)
print(numbers) # [1,2,3,4]
```

#### **extend(iterable)**

Dodaje **wiele elementów naraz**.

```
numbers.extend([5,6])
print(numbers) # [1,2,3,4,5,6]
```

#### **insert(index, x)**

Wstawia element w określone miejsce.

```
numbers.insert(1, 10)
print(numbers) # [1,10,2,3]
```

#### **remove(x)**

Usuwa **pierwsze wystąpienie elementu**. Jeśli element nie istnieje, zwraca błąd.

```
numbers.remove(2)
print(numbers) # [1,3]
```

#### **pop([index])**



Usuwa element i **zwraca go**.

```
numbers = [1,2,3,4]
last = numbers.pop()      # usuwa ostatni
print(last)               # 4
removedElementAtIndex = numbers.pop(1)  # usuwa element z indeksu
print(removedElementAtIndex)           # 2
print(numbers)                       # [1, 3]
```

### **clear()**

Usuwa wszystkie elementy.

```
numbers.clear()
print(numbers)      # []
```

### **index(x)**

Zwraca indeks pierwszego wystąpienia.

```
numbers = [1,2,3,4]
i=numbers.index(3)
print(i)          # 2
```

### **count(x)**

Liczy ile razy element występuje.

```
numbers = [1,1,2,3]
c=numbers.count(1)
print(c)          # 2
```

### **sort()**

Sortuje listę.

```
numbers = [3,1,2]
numbers.sort()
print(numbers)      # [1,2,3]
```

### **reverse()**

Odwraca kolejność.

```
numbers = [3,1,2]
numbers.reverse()
print(numbers)      # [2, 1, 3]
```

### **copy()**

Tworzy kopię listy.

```
numbers = [3,1,2]
new_list = numbers.copy()
numbers.append(6)
new_list.append(5)
print(numbers)      # [3, 1, 2, 6]
print(new_list)      # [3, 1, 2, 5]
```

**Set to nieuporządkowany zbiór bez duplikatów.**

$A = \{1,2,3\}$

### **add(x)**

Dodaje element. **Element 4 zostaje dodany do zbioru, ale nie wiadomo w jakiej kolejności.** Zbiór **nie ma uporządkowania**, więc nie można powiedzieć, że element jest „na końcu” ani „na początku”.

```
A = {1,2,3}
A.add(4)
print(A)           # {1, 2, 3, 4}
```

### **update(iterable)**

Dodaje wiele elementów. **Elementy zostaną dodane do zbioru, jeśli ich tam jeszcze nie było. Nie wiadomo w jakiej kolejności się pojawią przy wyświetleniu.**

```
A = {1, 2, 3}
A.update({7, 8})    # set
A.update((9, 10))   # tuple
A.update("ab")      # string → dodaje 'a' i 'b'
print(A)           # {1, 2, 3, 7, 8, 9, 10, 'b', 'a'}
```

### **remove(x)**

Usuwa element (błąd jeśli nie istnieje).

```
A = {1, 2, 3}
A.remove(2)
print(A)           # {1, 3}
```

### **discard(x)**

Usuwa element **bez błędu**.

```
A = {1, 2, 3}
A.discard(4)
print(A)           # {1, 2, 3}
```

### **pop()**

Usuwa **losowy element**. **Usuwa i zwraca usunięty jeden element ze zbioru. Nie wiadomo, który element zostanie usunięty, bo set jest nieuporządkowany.**

```
A = {1, 2, 3, 4}
x = A.pop()
print(x)           # 1
print(A)           # {2, 3, 4}
```

### **clear()**

Czyści zbiór.

```
A.clear()
print(A)           # set()
```

### **copy()**

```
A = {1, 2, 3, 4}
B = A.copy()
A.add(6)
B.add(7)
print(A)           # {1, 2, 3, 4, 6}
print(B)           # {1, 2, 3, 4, 7}
```

## Operacje zbiorów

### *union()*

Łączy zbiory.

A = {1,2}

B = {2,3}

```
print(A.union(B))    # {1,2,3}
```

### *intersection()*

Część wspólna.

A = {1,2}

B = {2,3}

```
print(A.intersection(B)) # {2}
```

### *difference()*

Różnica zbiorów.

A = {1,2}

B = {2,3}

```
print(A.difference(B))    # {1}
```

### *symmetric\_difference()*

Zwraca \*\*nowy zbiór zawierający elementy, które są w **A lub w B**, ale **nie w obu naraz**.

Formuła matematyczna:

$$A \triangle B = (A - B) \cup (B - A)$$

A = {1,2}

B = {2,3}

```
print(A.symmetric_difference(B)) # {1,3}
```

### *issubset()*

A.issubset(B)

- **Zwraca True**, jeśli każdy element zbioru A jest też w zbiorze B.
- **Zwraca False**, jeśli choć jeden element A nie występuje w B.

### Przykład

A = {1, 2}

B = {1, 2, 3, 4}

```
print(A.issubset(B))    # True
```

### *issuperset()*

A.issuperset(B)

- Zwraca **True**, jeśli **wszystkie elementy zbioru B są też w A**.
- Zwraca **False**, jeśli choć jeden element B nie występuje w A.

Innymi słowy: A jest **nadzbiorem** B.

### Przykład

A = {1, 2, 3, 4}

```
B = {2, 3}
print(A.issuperset(B))    # True
```

### *isdisjoint()*

```
A.isdisjoint(B)
```

```
A.isdisjoint(B)
```

- Zwraca **True**, jeśli **żaden element A nie występuje w B**.
- Zwraca **False**, jeśli jest **przynajmniej jeden wspólny element**.

Sprawdza czy zbiory nie mają części wspólnej.

### **Przykład – brak wspólnych elementów**

```
A = {1, 2, 3}
```

```
B = {4, 5, 6}
```

```
print(A.isdisjoint(B)) # True
```

## **DICTIONARY (dict)** Słownik to para klucz → wartość.

### **Podstawowy obiekt**

```
person = {
    "name": "Jan",
    "age": 30
}
```

### **get(key)**

Pobiera wartość bez błędu.

```
person = {
    "name": "Jan",
    "age": 30
}
```

```
print(person.get("name"))
```

### **keys()**

```
print(person.keys())    # dict_keys(['name', 'age']) zwraca wszystkie klucze.
```

### **values()**

```
print(person.values())  # dict_values(['Jan', 30]) zwraca wartości.
```

### **items()**

```
print(person.items())   # dict_items([('name', 'Jan'), ('age', 30), ('city', 'Warsaw')])
zwraca pary (key,value).
```

### **update()**

```
person = {
    "name": "Jan",
    "age": 30
}
person.update({"city": "Warsaw"})
```

```
print(person) # {'name': 'Jan', 'age': 30, 'city': 'Warsaw'}
```

### pop(key)

usuwa element

```
person = {
    "name": "Jan",
    "age": 30
}
person.pop("age")
print(person)      # {'name': 'Jan'}
```

### popitem()

usuwa **ostatnią** i zwraca co usuwa

```
person = {
    "name": "Jan",
    "first": "Kowalski",
    "age": 30,
    "gender": "M"
}

p = person.popitem()
print(p)      # ('gender', 'M')
print(person) # {'name': 'Jan', 'first': 'Kowalski', 'age': 30}
```

### setdefault()

Dodaje klucz jeśli nie istnieje.

```
person = {
    "name": "Jan",
    "age": 30
}

person.setdefault("country", "Poland")
print(person)      # {'name': 'Jan', 'age': 30, 'country': 'Poland'}
```

### clear()

```
person.clear()
print(person)      # {}
```

### copy()

```
person = {"name": "Jan", "age": 30, "city": "Warsaw"}

new_person = person.copy()
person.update({"city": "Warsaw2"})
new_person.update({"post-code": "12-333"})
```

```
print(person)      # {'name': 'Jan', 'age': 30, 'city': 'Warsaw2'}
print(new_person)  # {'name': 'Jan', 'age': 30, 'city': 'Warsaw', 'post-code': '12-333'}
```

new\_person to **nowy słownik**. Zmiany w new\_person **nie wpływają na person**, jeśli zmieniasz wartości bezpośrednio:

```
new_person["age"] = 31
print(person)      # {'name': 'Jan', 'age': 30, 'city': 'Warsaw'}
print(new_person)  # {'name': 'Jan', 'age': 31, 'city': 'Warsaw'}
```